



LayerCal

Deep Learning Parameter Calculator & AI Architecture Advisor

Technical Guide & Project Documentation

Chanjoong Kim

me@chanjoongx.com

Version 2.0.0 | February 2026

Project Information	
Live Demo	layercal.com
Tech Stack	React, Vite, Tailwind CSS
Deployment	Cloudflare Pages

Table of Contents

- 1. Project Overview
- 2. Core Features
- 3. Supported Layer Types
- 4. Calculation Formulas
- 5. Code Generation
- 6. AI Architecture Advisor
- 7. Technical Architecture
- 8. Performance Optimizations
- 9. Internationalization

1. Project Overview

What is LayerCal?

LayerCal is a browser-based tool designed to help ML engineers and researchers quickly estimate neural network complexity before training.

Key capabilities:

- Real-time calculation of parameters, memory usage, and FLOPs
- Auto-generated code for PyTorch, TensorFlow, and JAX
- AI-powered architecture generation from natural language descriptions
- Runs entirely in browser with no backend dependencies

Problem Statement

Before training deep learning models, practitioners need to understand resource requirements. Manual calculation of parameters for complex architectures is error-prone and time-consuming. Existing tools often require installation or rely on backend servers.

Target Users

- ML/AI researchers designing new architectures
- Students learning deep learning concepts
- Engineers estimating GPU memory requirements
- Teams planning training infrastructure

2. Core Features

Feature	Description
Parameter Counting	Accurate calculation for 14 layer types with configurable hyperparameters
Memory Estimation	Inference/Training modes, FP32/FP16/INT8 precision, optimizer overhead
FLOPs Calculation	Forward pass computational cost estimation
Code Generation	PyTorch, TensorFlow/Keras, JAX/Flax code output
AI Architecture Advisor	Natural language → validated layer stack via RAG + LLM
Drag & Drop UI	Intuitive layer palette with instant parameter updates
Dark Mode	System preference detection and manual toggle
Multi-language	8 languages: EN, KO, JA, ZH, ES, FR, DE, PT
PNG Export	One-click screenshot for documentation

Key Differentiators

- **Zero Backend:** All computation happens client-side
- **Smart Code Generation:** Consecutive layer grouping, context-aware normalization
- **AI-Assisted Design:** LLM-powered architecture generation with deterministic post-validation
- **Framework Parity:** Idiomatic code for each framework
- **Responsive Design:** Works on desktop and mobile

3. Supported Layer Types

LayerCal supports 14 common neural network layer types organized by category:

Category	Layer	Configurable Parameters
Embedding	Embedding	vocab_size, embedding_dim
Linear	Linear (Dense)	input_dim, output_dim, use_bias
Convolution	Conv2D	in_channels, out_channels, kernel_size, use_bias
Recurrent	LSTM	input_size, hidden_size, num_layers, bidirectional
Recurrent	GRU	input_size, hidden_size, num_layers, bidirectional
Attention	Transformer	d_model, num_heads, d_ff
Attention	Multi-Head Attention	d_model, num_heads
Normalization	BatchNorm	num_features
Normalization	LayerNorm	normalized_shape
Pooling	MaxPool2D	kernel_size
Pooling	AvgPool2D	kernel_size
Regularization	Dropout	rate (0.0-1.0)
Activation	ReLU	(no parameters)
Activation	Softmax	(no parameters)

Note: Dropout, ReLU, and Softmax have no trainable parameters but are included for architecture completeness.

4. Calculation Formulas

Parameter Count Formulas

Layer	Formula	Notes
Embedding	$V \times E$	$\text{vocab_size} \times \text{embedding_dim}$
Linear	$(I \times O) + O$	with bias
Conv2D	$(C_{in} \times C_{out} \times K \times K) + C_{out}$	with bias
LSTM	$4 \times (I \cdot H + H^2 + 2H) \times L \times \text{dir}$	4 gates, 2 biases per gate
GRU	$3 \times (I \cdot H + H^2 + 2H) \times L \times \text{dir}$	3 gates, 2 biases per gate
Transformer	$(12 \times d^2 + 13d) \times L$	assuming $d_{ff} = 4d$
BatchNorm	$2 \times F$	gamma + beta
LayerNorm	$2 \times N$	gamma + beta
Attention	$4 \times (d^2 + d)$	Q, K, V, O projections with bias

Legend: V = vocab size, E = embedding dim, I = input dim, O = output dim, C = channels, K = kernel size, H = hidden size, L = num layers, d = model dim, F = features, N = normalized shape, dir = direction (1 for unidirectional, 2 for bidirectional)

Memory Estimation

Inference Mode

Memory = Parameters × Bytes per Parameter

Precision	Bytes/Param	Use Case
FP32	4	Full precision training/inference
FP16	2	Mixed precision training, GPU inference
INT8	1	Quantized inference, edge deployment

Training Mode

Training requires additional memory for gradients and optimizer states:

Optimizer	Multiplier	Reason
SGD	2×	Parameters + Gradients
SGD + Momentum	3×	+ Momentum buffer
Adam	4×	+ First moment + Second moment

FLOPs Estimation (Forward Pass)

Layer	FLOPs Formula
Linear	$2 \times I \times O \times B$ (multiply-add)
Conv2D	$2 \times C_{in} \times C_{out} \times K^2 \times H_{out} \times W_{out} \times B$
LSTM (per timestep)	$8 \times H^2 + 8 \times I \times H$ (per layer, per direction)
Attention	$4 \times S^2 \times d + 2 \times S \times d^2$ (Q·K, softmax, ·V, projection)

5. Code Generation

LayerCal generates idiomatic, runnable code for three major frameworks. The code generator includes smart features that produce cleaner output than simple concatenation.

Smart Features

Feature	Description	Example
Layer Grouping	Groups consecutive identical layers	3× Transformer → TransformerEncoder(num_layers=3)
Context-aware BatchNorm	Selects 1D/2D based on previous layer	After Conv2D → BatchNorm2d
Semantic Naming	Meaningful variable names	self.embed, self.fc1, self.conv
Framework Idioms	Follows each framework's conventions	TF: Sequential vs Functional API

PyTorch Output Example

```
import torch
import torch.nn as nn

class GeneratedModel(nn.Module):
    """Generated by LayerCal - layercal.com"""

    def __init__(self):
        super().__init__()
        self.embed = nn.Embedding(50000, 512)
        self.transformer_layer = nn.TransformerEncoderLayer(
            d_model=512, nhead=8,
            dim_feedforward=2048, batch_first=True
        )
        self.transformer = nn.TransformerEncoder(
            self.transformer_layer, num_layers=6
        )
        self.fc = nn.Linear(512, 10)

    def forward(self, x):
        x = self.embed(x)
        x = self.transformer(x)
        x = self.fc(x[:, 0])
        return x
```

Framework Support Matrix

Layer	PyTorch	TensorFlow	JAX/Flax
Embedding	✓	✓	✓
Linear	✓	✓	✓

Conv2D	✓	✓	✓
LSTM	✓	✓ (partial)	○
GRU	✓	✓ (partial)	○
Transformer	✓	○	○
Attention	✓	✓	○
BatchNorm	✓	✓	△ (partial)
LayerNorm	✓	✓	✓
Pooling	✓	✓	○
Dropout	✓	✓	✓
Activations	✓	✓	✓

✓ = Full support, ○ = Placeholder/manual implementation needed

6. AI Architecture Advisor

The AI Architecture Advisor lets users describe a model in plain English and receive a complete, validated layer stack that can be applied to the canvas with one click.

System Architecture

The advisor follows a Retrieval-Augmented Generation (RAG) pattern, implemented entirely client-side with no middleware or backend:

Stage	Component	Description
1. Retrieval	architectureKB.js	Keyword matching against 12 reference architectures (LeNet, VGG, MobileNet, BERT, GPT, etc.)
2. Augmentation	ragPipeline.js	Builds a system prompt with layer schema, valid parameter ranges, and top-3 matched references
3. Generation	llmClient.js	Calls the selected LLM provider API directly from the browser
4. Validation	ragPipeline.js	Parses JSON, snaps params to valid values, fixes cross-layer dimension mismatches

Supported Providers

Users bring their own API key (BYOK). Keys are stored in the browser's localStorage per provider and never transmitted to any server other than the chosen provider's API endpoint.

Provider	Default Model	Free Tier
Google Gemini	gemini-2.5-flash-lite	1,000 requests/day
OpenAI	gpt-4o-mini	Requires billing
Anthropic Claude	claude-3-5-haiku	Requires billing

Post-Processing & Validation

LLM outputs are non-deterministic, so a deterministic validation pipeline ensures every generated architecture is runnable:

Step	What It Does	Example
JSON Extraction	Strips markdown fences, thinking blocks, explanatory text; extracts the outermost JSON array	<code><think>...</think>[...]</code> → <code>[...]</code>
Type Filtering	Removes layers not in the 14 supported types	<code>flatten</code> → skipped
Parameter Snapping	Snaps numeric values to nearest valid option in the dropdown	<code>kernel_size: 4</code> → <code>3</code>
Cross-layer Fix	Tracks output dimensions through the stack and fixes mismatches	<code>Conv2D(out=32)</code> → <code>BN(64)</code> → <code>Conv2D(out=32)</code> → <code>BN(32)</code>

Upstream Reconciliation	When downstream requires a specific dim, walks backward through passthrough layers to fix the source	Conv2D (out=32) → ReLU → BN (128) → both fixed to 64
-------------------------	--	--

Knowledge Base

The retrieval step matches user queries against 12 architecture templates:

Architecture	Category	Approx. Parameters
LeNet-5	CNN	~62K
Simple CNN Classifier	CNN	~1.2M
MobileNet-style (Lite)	CNN	~85K
VGG-style Network	CNN	~14M
Convolutional Autoencoder	Autoencoder	~200K
LSTM Text Classifier	RNN	~3.5M
GRU Sequence Model	RNN	~1.8M
Small Transformer Encoder	Transformer	~7M
BERT-style Encoder	Transformer	~38M
GPT-style Decoder	Transformer	~50M
MLP Classifier	Tabular	~400K
Self-Attention Classifier	NLP	~4M

7. Technical Architecture

Tech Stack

Layer	Technology	Purpose
UI Framework	React 18	Component-based UI with hooks
Build Tool	Vite	Fast HMR, optimized production builds
Styling	Tailwind CSS	Utility-first CSS framework
Components	shadcn/ui	Accessible, customizable primitives
Icons	Lucide React	Consistent icon set
Testing	Vitest	Unit tests for calculations and RAG pipeline
State	React useState + localStorage	Client-side persistence
Hosting	Cloudflare Pages	Global CDN, automatic HTTPS
Analytics	Google Analytics 4	Usage tracking

Project Structure

```
layercal/
├── src/
│   ├── components/
│   │   ├── LayerCal.jsx      # Main app component
│   │   ├── AIAdvisor.jsx     # AI Advisor modal UI
│   │   └── ui/               # shadcn primitives
│   ├── config/
│   │   ├── layerTypes.js     # Layer definitions, formulas
│   │   ├── translations.js   # i18n strings (8 languages)
│   │   └── architectureKB.js # 12 reference architectures
│   ├── utils/
│   │   ├── llmClient.js      # Multi-provider LLM client
│   │   ├── ragPipeline.js    # RAG: retrieval, prompting, validation
│   │   ├── codeGenerator.js
│   │   ├── imageExport.js
│   │   └── localStorage.js
│   └── __tests__/
│       ├── layerTypes.test.js # 44 tests: params, FLOPs, memory
│       └── ragPipeline.test.js # 41 tests: parsing, snapping, cross-layer
├── public/
│   └── og-image.png
├── index.html
└── package.json
```

8. Performance Optimizations

LayerCal implements several React optimization patterns to ensure smooth interaction even with many layers:

Technique	Implementation	Benefit
useMemo	LAYER_TYPES, totalParams, modelSizeMB	Prevents recalculation on every render
useCallback	All event handlers (addLayer, deleteLayer, etc.)	Stable function references
Functional setState	setModelLayers(prev => ...)	Avoids stale closure issues
Conditional Rendering	Language menu, modals	Reduces DOM nodes
Event Delegation	Single handler for layer palette	Fewer event listeners

Bundle Size: Production build with Vite produces optimized chunks. Tree shaking removes unused code from dependencies. Total bundle size is under 200KB gzipped.

9. Internationalization

LayerCal supports 8 languages: English, Korean, Japanese, Chinese (Simplified), Spanish, French, German, and Portuguese.

Translations are stored as nested objects in `translations.js`. Current language is persisted to `localStorage`. Layer descriptions, UI labels, tooltips, and AI Advisor interface text are all translated.

Code	Language
en	English
ko	한국어
ja	日本語
zh	中文
es	Español
fr	Français
de	Deutsch
pt	Português

LayerCal developed by Chanjoong Kim

Email: me@chanjoongx.com | GitHub: [@chanjoongx](https://github.com/chanjoongx)